

# Análise de Código Java

Busca Binária — BinarySearch.java

## 1. Objetivo Geral

O código implementa o algoritmo de **busca binária** em um array de inteiros ordenado. O método *binarySearch* recebe o array e um valor-alvo, retornando o índice do elemento encontrado ou **-1** caso não exista. O método *main* demonstra três chamadas: busca por um elemento presente no meio (7), por um ausente (6) e pelo último elemento (15).

## 2. Lógica e Algoritmo

A cada iteração do laço *while*, o algoritmo calcula o índice médio ( $mid = left + (right - left) / 2$ ) e compara o elemento central com o alvo. Se igual, retorna o índice imediatamente. Se o alvo for maior, descarta a metade esquerda avançando *left*; se menor, descarta a metade direita recuando *right*. O processo repete-se até encontrar o valor ou esgotar o intervalo.

```
int mid = left + (right - left) / 2;
if (arr[mid] == target) return mid;
else if (arr[mid] < target) left = mid + 1;
else right = mid - 1;
```

## 3. Conceitos Java Envolvidos

O código utiliza apenas recursos básicos da linguagem: métodos estáticos (*static*), arrays primitivos (*int[]*), laço iterativo *while* e retorno antecipado (*return*). Não há uso de generics, coleções, recursão ou threads — a solução é propositalmente simples e sem dependências externas além do pacote *java.util* (importado, mas não utilizado diretamente).

## 4. Complexidade

| Dimensão                | Valor    | Justificativa                                         |
|-------------------------|----------|-------------------------------------------------------|
| Tempo — Melhor caso     | O(1)     | Alvo encontrado na primeira comparação (mid)          |
| Tempo — Médio/Pior caso | O(log n) | Intervalo é halvado a cada iteração                   |
| Espaço                  | O(1)     | Apenas variáveis escalares; sem estruturas auxiliares |

## 5. Decisões de Design e Limitações

O cálculo  $mid = left + (right - left) / 2$  evita overflow inteiro em relação à forma ingênua  $(left + right) / 2$  — boa prática para arrays grandes. A principal limitação é a **pré-condição**

**silenciosa:** o array *deve* estar ordenado; caso contrário, o resultado é incorreto sem qualquer aviso. Além disso, a implementação não suporta arrays de objetos nem duplicatas (retorna apenas um índice). Para uso em produção, recomenda-se *Arrays.binarySearch()* da biblioteca padrão Java.